Efficient SAT Techniques for Relative Encoding of Permutations with Constraints

Miroslav N. Velev* and Ping Gao

Aries Design Automation
Chicago, IL 60660, U.S.A.
miroslav.velev@aries-da.com
http://www.aries-da.com

Abstract. We present new techniques for relative SAT encoding of permutations with constraints, resulting in improved scalability compared to the previous approach by Prestwich, when applied to searching for Hamiltonian cycles. We observe that half of the ordering variables and two-thirds of the transitivity constraints can be eliminated. We exploit minimal enumeration of transitivity, based on 12 triangulation heuristics, and 11 heuristics for selecting the first node in the Hamiltonian cycle. We propose the use of inverse transitivity constraints. We achieve 3 orders of magnitude average speedup on satisfiable random graphs from the phase transition region, 2 orders of magnitude average speedup on unsatisfiable random graphs, and up to 4 orders of magnitude speedup on satisfiable structured graphs from the DIMACS graph coloring instances.

1 Introduction

We investigate efficient SAT techniques for solving of problems that can be reformulated as permutations with constraints, based on the relative SAT encoding [21], thus exploiting the tremendous advances in the speed and capacity of SAT solvers without reimplementing the same optimizations in specialized tools for specific problems. Particularly, we do an in-depth study of Hamiltonian Cycle Problems (HCPs)—where the goal is to find a route in a graph by visiting each node exactly once and returning to the starting node—a known class of hard combinatorial problems, classified as NP-complete (p. 199 of [9]). Another hard combinatorial problem, quasigroup completion [1, 11, 16, 28], can be reformulated as multiple permutations with constraints; it has applications to design of experiments, and wavelength routing in switches on optical networks. Other combinatorial problems that can be viewed as permutations with constraints are discussed in [2, 12]. At NASA, problems that can be reformulated as permutations of tasks, subject to additional constraints, arise in preparing sites for human habitation [8]. The efficient encoding of real-world problems as equivalent SAT formulas is a challenge identified by Kautz and Selman [17].

The first method proposed for HCP solving by translation to SAT was based on the absolute SAT encoding of permutations [15], and was used in [2, 13, 21]. However,

* Corresponding author.

A. Nicholson and X. Li (Eds.): AI 2009, LNAI 5866, pp. 517–527, 2009.
© Springer-Verlag Berlin Heidelberg 2009

those papers present results for graphs with at most 24 nodes. Recently, we did an in-depth study of techniques to improve the scalability of the absolute SAT encoding for permutations with constraints, when applied to solving HCPs [29]. We found that the above method for absolute SAT encoding of permutations does not scale for a suite with 100 graphs of 30 nodes each, generated in the phase-transition region [3, 7, 27]. We proposed new techniques that improved the scalability of the absolute encoding, and resulted in at least 4 orders of magnitude average speedup when solving HCPs for both satisfiable and unsatisfiable benchmarks. However, the new techniques scaled only for graphs with up to 95 nodes.

In this paper, we extend the relative SAT encoding of permutations with constraints [21], and present techniques to improve its scalability when solving HCPs. The paper makes four contributions: 1) the observation that half of the ordering variables and two-thirds of the transitivity constraints can be eliminated in the relational encoding of permutations with constraints [21]; 2) the use of minimal enumeration of transitivity, based on 12 triangulation heuristics, and 11 heuristics for selecting the first node in the Hamiltonian cycle; 3) the concept of inverse transitivity constraints; and 4) experimental results showing 3 orders of magnitude average speedup on satisfiable random graphs from the phase transition region, 2 orders of magnitude average speedup on unsatisfiable random graphs, and up to 4 orders of magnitude speedup on satisfiable structured graphs from the DIMACS graph coloring instances [5].

2 Background

In the *relative encoding* [21], represented are the relative positions of objects with respect to each other in a permutation. The motivation for this encoding is the observation that in the absolute encoding of permutations, if node v is to be moved forward (backward) in a permutation, then all nodes between the old and the new positions of v will also have to be moved, which will result in value changes for the Boolean variables encoding the placement of all those nodes. In contrast, in the relative encoding, we only need to change the relative position of node v with respect to the rest of the nodes by changing the values of some or all of the Boolean variables used for node v, while keeping the rest of the nodes in their original relative positions, i.e., keeping the values assigned to the Boolean variables encoding the relative placement of those nodes, thus significantly reducing the number of Boolean variables whose values have to be changed, and so significantly reducing the corresponding work of the SAT solver. Since Prestwich [21] presented constraints for Hamiltonian paths, while our focus is on Hamiltonian cycles that require additional constraints to ensure that the first and last nodes in the permutation are neighbors in the graph, we present next an extension of his formulation for Hamiltonian cycles.

Two types of Boolean variables are introduced—successor and ordering variables. The *successor variables* encode the constraints that exactly one out of a node's neighbors is selected to be its successor in the permutation, and exactly one is selected to be its predecessor, based on the direct encoding [4]. A successor Boolean variable s_{ij} is introduced for every ordered pair of nodes (v_i, v_j) that are neighbors in the graph, i.e., 2 such variables are required for each edge—one variable for each of the two

orderings of the two nodes of the edge—such that s_{ij} is true iff node v_j appears immediately after v_i in the permutation that represents the ordering of the nodes in the Hamiltonian cycle.

The constraints for the successor variables are:

- each node v_i has at least one successor from its neighbors: $s_{ij} \vee s_{ik} \vee \ldots$, where the neighbors of v_i are nodes v_j, v_k, ... ;
- each node v_i has at most one successor from its neighbors: $\neg s_{ij} \vee \neg s_{ik}$, for every pair of neighbors v_j and v_k of node v_i;
- each node v_i is the successor of at least one of its neighbors: $s_{ji} \vee s_{ki} \vee \ldots$, where the neighbors of v_i are nodes v_j, v_k, ... ; and
- each node v_i is the successor of at most one of its neighbors: $\neg s_{ji} \vee \neg s_{ki}$, for every pair of neighbors v_j and v_k of node v_i.

The *ordering variables* encode the relative ordering of the nodes in the permutation that represents the Hamiltonian cycle, starting from the node selected to be first, v_f, and ending with the first node's neighbor selected to be the last node in the cycle, v_l. An ordering Boolean variable o_{ij} is introduced only for ordered pairs of two different nodes (v_i, v_j), where $i \neq j$. The ordering variable o_{ij} is defined to be true iff node v_i appears before node v_j in the permutation. The ordering variables satisfy several properties:

- *transitivity*: $o_{ij} \wedge o_{jk} \Rightarrow o_{ik}$, i.e., $\neg o_{ij} \vee \neg o_{jk} \vee o_{ik}$, for all permutations of 3 nodes v_i, v_j, and v_k, such that $i \neq j$, $j \neq k$, and $i \neq k$;
- *antisymmetry*: $\neg(o_{ij} \wedge o_{ji})$, or equivalently $\neg o_{ij} \vee \neg o_{ji}$, i.e., it is impossible for node v_i to be before node v_j and for node v_j to be before node v_i simultaneously;
- the first node v_f precedes all others: o_{fi}, for all $i \neq f$; and
- the first node's neighbor v_l selected to be the last node in the cycle succeeds all others: $s_{lf} \Rightarrow o_{il}$, or equivalently $\neg s_{lf} \vee o_{il}$, for all $i \neq f$, $i \neq l$, and all neighbors v_l of the first node v_f.

Finally, the relationship between successor and ordering variables is: $s_{ij} \Rightarrow o_{ij}$, or equivalently $\neg s_{ij} \vee o_{ij}$, i.e., if node v_i has its neighbor v_j selected to be v_i's successor along the Hamiltonian cycle, then the ordering variable o_{ij} is true.

Prestwich [21] enforced transitivity for all possible permutations of three nodes. However, this results in $n \times (n - 1)$ ordering variables and $n \times (n - 1) \times (n - 2)$ transitivity constraints for a graphs with n nodes, i.e., in a prohibitive number of transitivity constraints for large graphs. We refer to this method as *exhaustive enumeration of transitivity*, and will present improvements to it in Sect. 3 and 4.

3 Eliminating Half of the Ordering Variables and Two-Thirds of the Transitivity Constraints

We exploit the observation that the ordering variables o_{ij} and o_{ji} are complements of each other, since when node v_i precedes node v_j then o_{ij} is true and o_{ji} is false by definition, and vice versa—when node v_j precedes node v_i then o_{ij} is false and o_{ji} is true by definition. Thus, we can introduce an ordering variable o_{ij} only if the node

index i is less than the node index j, and replace the ordering variable o_{ji} with $\neg o_{ij}$. This leads to the following lemma:

LEMMA 1. *For a triple of nodes* $\{v_i, v_j, v_k\}$, *whose indices are ordered* $1 \le i < j < k \le n$, *where n is the number of nodes in the graph, after half of the ordering variables are eliminated by introducing an ordering variable* o_{pq} *only for an ordered pair of nodes* (v_p, v_q) *where the node indices satisfy* $1 \le p < q \le n$, *and replacing the ordering variable* o_{qp} *with* $\neg o_{pq}$, *then the six transitivity constraints that are introduced for the triple of nodes* $\{v_i, v_j, v_k\}$ *in exhaustive enumeration of transitivity will reduce to two transitivity constraints:* (1) $\neg o_{ij} \vee \neg o_{jk} \vee o_{ik}$; *and* (2) $o_{ij} \vee o_{jk} \vee \neg o_{ik}$.

Thus, half of the ordering variables, two-thirds of the transitivity constraints, and all the antisymmetry constraints (that evaluate to true now) are eliminated. This results in fewer decisions to be made by the SAT solver when solving the resulting CNF formula, as well as reduces significantly the *Boolean Constraint Propagation* (BCP)—the iterative process of assigning values to literals as implied by clauses that have become unit, i.e., where all literals but one have assigned values, and those values are false, so that the only literal without a value has to be set to true in order for the clause to be satisfied. BCP takes up to 90% of the execution time of SAT solvers [19].

4 Minimal Enumeration of Transitivity

Exhaustive transitivity requires $n \times (n - 1)$ ordering variables and $n \times (n - 1) \times (n - 2)$ transitivity constraints, where n is the number of nodes in the graph—see Sect. 2. Eliminating half of the ordering variables as described in Sect. 3 and applying Lemma 1 results in $n \times (n - 1) / 2$ ordering variables and $n \times (n - 1) \times (n - 2) / 3$ transitivity constraints. In this section we aim to further reduce the number of ordering variables and transitivity constraints by introducing only a minimal set of such variables and constraints in a way that the property of transitivity will be enforced for the *required ordering variables*—those that are used in the constraints in Sect. 2, except for the transitivity constraints.

We introduce the concept of a *relational graph* that has the same set of nodes as the original graph that we want to find a Hamiltonian cycle in, but an extended set of edges, such that a pair of nodes is connected with an edge in the relational graph iff an ordering variable is introduced for that pair of nodes. We will use the relational graph to efficiently enforce the property of transitivity for the required ordering variables. These variables determine three types of edges that must be present in the relational graph:

1. each edge from the original graph, since the relationship between successor and ordering variables requires that each edge in the original graph should have an associated ordering variable;
2. given a choice for a first node, v_f, an edge between v_f and each of the other $n - 1$ nodes in the relational graph, because of the constraints that v_f must precede all other nodes in the permutation; and

3. given a choice for a first node, v_f, an edge between each of v_f's neighbors and the other $n - 2$ nodes in the relational graph besides v_f, because of the conditions that when that neighbor is selected to be the last node in the permutation, then the other nodes must precede that neighbor in the permutation (that v_f precedes the last node is already ensured with constraints that resulted in edges of type 2 above).

We refer to the resulting graph as *base relational graph*, and designate it R_B. Because of the second and third types of edges that depend on the choice of the first node, v_f, the base relational graph must be constructed after the first node in the Hamiltonian cycle is selected. Note that the base relational graph R_B is different from the original graph for which we are searching for a Hamiltonian cycle, and is used for an efficient relative SAT encoding of the node permutation.

Besides the three types of required edges in R_B that were described, we want to introduce additional edges such that the resulting graph becomes *chordal* [22]. That is, the graph has the property that every cycle of length greater than three has a chord—an edge joining two nodes that are not adjacent in the cycle. Chordal graphs are also called *triangulated graphs*. We can then generate a sufficient set of transitivity constraints by enforcing transitivity for each triangle in the resulting graph. Computing a minimal triangulation consists of embedding a given graph into a triangulated graph by adding a set of edges called *a fill*. Finding a fill that is minimum is NP-complete [30]. However, there are good heuristic solutions. The ones that we explored are described next.

Our triangulation method proceeds as a series of elimination steps, starting with graph $G_0 = R_B$. On elimination step i, we create a new graph G_i that is identical to G_{i-1}, except that some vertex v_m and its incident edges are removed, and new edges are possibly added in the following way: for every pair of distinct vertices v_j and v_k such that G_{i-1} contains the edges (v_m, v_j) and (v_m, v_k), we add the edge (v_j, v_k) to G_i if this edge does not already exist. This process continues until we reach an empty graph G_n. Let R_T be the relational graph obtained from R_B after extending it with all edges added during the elimination process. It can be shown that R_T is a chordal graph [22]. To choose which node v_m to eliminate on step i, we implemented the following 12 triangulation heuristics that select the node with the current minimum degree, and if there are several nodes with such degree, the tie is broken as follows: *t1*—break the tie based on a lesser sum of neighbors' degrees; *t2*—break the tie based on a greater sum of neighbors' degrees; *t3*—break the tie based on a lesser fill (i.e., additional edges) that will be added at this step of the elimination; *t4*—break the tie based on a greater fill that will be added at this step of the elimination; *t5*—break the tie based on a lesser original degree of the node in R_B; *t6*—break the tie based on a greater original degree of the node in R_B; *t7*—break the tie based on a lesser number of additional triangles created in R_T that include node v_m, and edges from G_{i-1} and those added at step i; *t8*—break the tie based on a greater number of additional triangles created in R_T that include node v_m, and edges from G_{i-1} and those added at step i; *t9*—select the node that will result in a minimum fill at this step of the elimination, and break ties by selecting the first such node in the graph description; *t10*—select the node that will result in a minimum fill at this step of the elimination, and break ties randomly; *t11*—select the node that will result in a minimal number of additional triangles created in R_T that include node v_m, and edges from G_{i-1} and those added at step i, and break ties

by selecting the first such node in the graph description; and *t12*—select the node that will result in a minimal number of additional triangles created in R_T that include node v_m, and edges from G_{i-1} and those added at step i, and break ties randomly.

We use the triangles in R_T to enforce transitivity of the ordering variables. When alhe optimizations from Sect. 3, we only introduce one ordering variable per edge in R_T and the two transitivity constraints per triangle—see Lemma 1.

Triangulation heuristic *t10* was also explored by Kjaerulff [18], who found it to performance when triangulating belief networks in knowledge representation. He called that heuristic minimum fill. Rose [23] refers to it as minimum deficiency heuristic, and references other papers [25, 26] in which it is assumed that this heuristic produces a near optimal node-elimination ordering.

5 Selecting the First Node for the Hamiltonian Cycle

We exploit the observation that when the search is for a Hamiltonian cycle, as opposed to a Hamiltonian path, the first node in the cycle (i.e., permutation) can be selected in any way, since all nodes have to be included in the cycle, and if a cycle exists then all nodes are symmetrical in it. Thus, if a cycle does not exist when a particular node is selected as a first node, then a cycle would not exist when any other node is selected as a first node. That is, the first node can be chosen in any way because of the symmetry of a solution, and the general formulation of the relative encoding in Sect. 2.

We implemented the following 11 heuristics for selecting the first node: *f1*—choose the first node in the graph description; *f2*—choose the first node of max. degree in the graph description; *f3*—choose the first node of min. degree in the graph description; *f4*—choose the first node of average degree in the graph description; *f5*—random choice; *f6*—choose a node of max. degree, and break ties based on a lesser sum of neighbors' degrees; *f7*—choose a node of max. degree, and break ties based on a greater sum of neighbors' degrees; *f8*—choose a node of average degree, and break ties based on a lesser sum of neighbors' degrees; *f9*—choose a node of average degree, and break ties based on a greater sum of neighbors' degrees; *f10*—choose a node of min. degree, and break ties based on a lesser sum of neighbors' degrees; and *f11*—choose a node of min. degree, and break ties based on a greater sum of neighbors' degrees. In contrast, Prestwich [21] selected the first node for a Hamiltonian path (he was searching for Hamiltonian paths, as opposed to cycles) to be the first node in the graph description.

6 Inverse Transitivity Constraints

We consider triples of nodes v_i, v_j, and v_k, such that v_i and v_j are neighbors in the original graph. Then, if node v_j is selected to be the successor of node v_i in the permutation, as indicated by the successor Boolean variable s_{ij} being true, and node v_i precedes node v_k in the permutation, as indicated by the ordering Boolean variable o_{ik} being true, it follows that node v_j must also precede node v_k in the permutation, i.e., o_{jk} must be true, since v_j is the closest node that is after v_i and thus v_k must be after v_j,

i.e., $s_{ij} \wedge o_{ik} \Rightarrow o_{jk}$, or equivalently $\neg s_{ij} \vee \neg o_{ik} \vee o_{jk}$ (see Fig. 1.a). We call these constraints *forward inverse transitivity constraints*. Note that they are different from the regular transitivity constraints that state that if v_i is before v_j, and v_j is before v_k, then v_i is before v_k.

We can also define a variant of these constraints for the case when node v_k is before node v_j, as indicated by the ordering Boolean variable o_{kj} being true, such that node v_j is selected to be the successor of node v_i in the permutation, as indicated by the successor Boolean variable s_{ij} being true. Then, it follows that node v_k must be before v_i, i.e., o_{ki} must be true, since v_i is the closest node that is before v_j by being the predecessor of v_j and thus v_k must be before v_i, i.e., $s_{ij} \wedge o_{kj} \Rightarrow o_{ki}$, or equivalently $\neg s_{ij} \vee \neg o_{kj} \vee o_{ki}$ (see Fig. 1.b). We call these constraints *backward inverse transitivity constraints.*

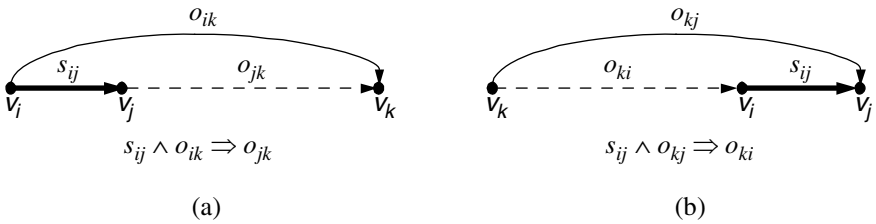

Fig. 1. Illustration of: (a) forward inverse transivity; (b) backward inverse transivity

When the optimizations from Sect. 3 are also applied, only two of the above four ordering variables will actually be introduced, while the other two will be each replaced with the negation of the introduced ordering variable for the same pair of nodes. In our tool, we imposed the inverse transitivity constraints for every ordered pair of neighboring nodes (v_i, v_j) from the original graph, and for every node v_k that is different from v_i and v_j, such that the ordering variables for v_k with respect to both v_i and v_j have been introduced by corresponding edges in the triangulated relational graph R_T (see Sect. 4). Note that these constraints are optional, and that we can use only a subset of them.

7 Results

The experiments were conducted on a Dell Precision T7400 workstation with two 3.2-GHz quad-core Intel Xeon processors, 32 GB of 800-MHz memory, and running Red Hat Enterprise Linux v5.3. Only one CPU core was used for each experiment. We started with satisfiable benchmarks (guaranteed to have Hamiltonian cycles) by generating suites of 100 graphs from the phase-transition region [3] that satisfy the ratio $e / (n \log n) = 1$, where e is the number of edges and n the number of nodes in the graph, since that ratio was shown to result in the hardest instances for backtrack search algorithms [7, 27]. We used the Hamiltonian-graph generator by Vandegriend and Culberson[1] [27]—that was also used in all previous papers on SAT-based solving

[1] http://web.cs.ualberta.ca/~joe/Theses/HCarchive/main.html

of HCPs [2, 13, 21, 29]. We produced 9 suites of 100 random graphs each from the phase-transition region, with the following number of nodes in the graphs of a suite, respectively—20, 30, 50, 100, 150, 200, 250, 300, and 350—for a total of 900 graphs.

We compared the SAT solvers siege [24], BerkMin [10], minisat v1.14 [6], tinisat [14], and rsat v3.1 [20], of which rsat v3.1 outperformed the rest by 2× or more, and so was used for the experiments discussed next. Fig. 2 presents the total CPU times for the 9 suites, plotted on a logarithmic scale, for the previous translation, and the best strategy for each of the proposed optimizations. As can be seen, the speedup is increasing with the size of the graphs. For the suite with the largest graphs of 350 nodes, eliminating half of the ordering variables and two-thirds of the transitivity constraints resulted in approximately an order of magnitude speedup, followed by another order of magnitude speedup from minimal transitivity with heuristic *t4* and selecting the first node in the Hamiltonian cycle with heuristic *f4*, and then another order of magnitude speedup from also exploiting inverse transitivity constraints combined with heuristics *f1* and *t9*.

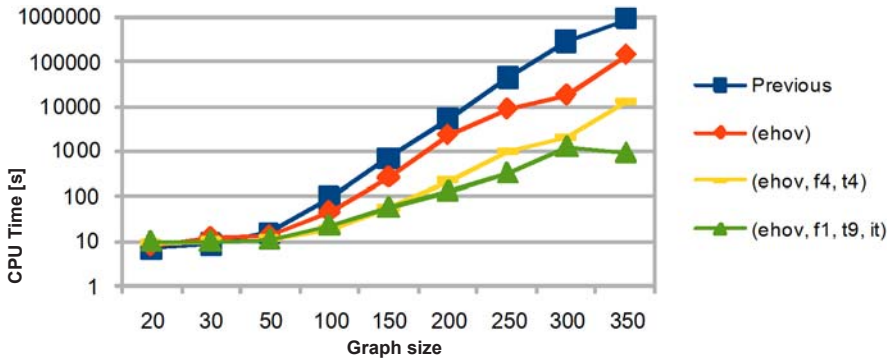

Fig. 2. Total CPU time [s] on a logarithmic scale for all 100 graphs in each of the 9 suites of satisfiable benchmarks from the phase-transition region

Table 1 shows detailed statistics for the four strategies from Fig. 2 for the suite of 100 graphs with 350 nodes. Strategy (*ehov*, *f1*, *t9*, *it*) resulted in efficient pruning of the solution space, reduced the number of conflicts and decisions by 3 orders of magnitude, and produced an average speedup of 1,071×, i.e., *3 orders of magnitude average speedup* computed based on the total time for the suite of 100 graphs with 350 nodes, relative to the previous translation. The individual speedup was up to 4 orders of magnitude. *Note that the reduction in the number of clauses and file size by approximately 30× will allow scalability for significantly larger instances.*

Although inverse transitivity constraints increased the number of clauses by 19% on average, they produced speedup due to the resulting pruning of the solution space, and eliminated wasteful work of the SAT solver by preventing it from exploring assignments that are guaranteed to not be part of solutions. This coincides with conclusions from previous work that adding redundant constraints to a CNF formula may make it easier to solve, e.g., see [1, 11, 16, 28].

Table 1. Details of the four strategies in Fig. 2 on the suite of 100 graphs with 350 nodes

Strategy	CNF Formula Average Statistics			SAT Solving Average Statistics		Average CPU Time per Graph [s]				Total CPU Time for Entire Suite [s]	Speedup
	Variables	Clauses	File Size [MB]	Conflicts	Decisions	Trans-late	SAT Solve	Check	Total for 3 Steps		
Previous	124,818	$42{,}657\times10^3$	905	119,758	1,402,382	14.2	9,425	0.03	9,439	943,924	———
(*ehov*)	63,743	$14{,}196\times10^3$	284	53,748	629,956	5.4	1,424	0.02	1,429	142,919	6.6×
(*ehov*, *f4*, *t4*)	18,596	$1{,}339\times10^3$	25	46,638	508,685	6.5	124	0.4	131	13,053	72×
(*ehov*, *f1*, *t9*, *it*)	18,544	$1{,}593\times10^3$	30	34	875	7.1	1.3	0.4	8.8	882	1,071×

To conduct experiments with unsatisfiable benchmarks that do not have Hamiltonian cycles, we generated such instances by implementing a program to rewrite each of the graphs with Hamiltonian cycles used for the presented experiments by injecting structures that make it impossible to find a Hamiltonian cycle in the resulting graph. We used two such transformations. First, selecting a random node of degree greater than 2, and modifying each of its neighbors to make them of degree 2, thus ensuring that only two of the neighbors can be on a path that includes the selected node, and so making it impossible to find a Hamiltonian cycle in the resulting graph. Second, selecting a random number of nodes up to 60% of the nodes in the graph, but at least 7 nodes, and converting them to form a chain of k cycles of 4 nodes each, where each of these cycles is a sequence of four nodes (n_1, n_2, n_3, n_4), such that node n_1 for the first cycle in the chain is connected to a random node in the portion of the graph that was not used for the chain, and node n_3 in each cycle is used as node n_1 for the next cycle in the chain, except in the last cycle of the chain, where node n_3 is connected to a random node in the portion of the graph that was not used for the chain. Since nodes n_2 and n_4 in each cycle are of degree 2, it is impossible to include all four nodes in such a cycle in a Hamiltonian cycle for the entire graph. Furthermore, this chain has a total of 2^k paths between node n_1 in the first cycle and node n_3 in the last cycle, so that a backtrack search algorithm may be affected negatively by the exponential number of such paths. Thus, each of the resulting 1,800 graphs was guaranteed to not have a Hamiltonian cycle. We found that strategy (*ehov*, *f1*, *t9*, *it*) resulted in up to 2 orders of magnitude of average speedup, compared to the previous translation. The speedup is again increasing with the size of the graphs.

To evaluate the benefit from these optimizations on structured benchmarks, we ran experiments with the DIMACS graph coloring instances [5]—see Table 2. The best was strategy (*ehov*, *f4*, *t4*, *it*), resulting in up to *4 orders of magnitude speedup.* After we filtered the graphs that were trivially unsatisfiable because of nodes of degree 0 or 1, all of the remaining DIMACS graphs had Hamiltonian cycles.

Table 2. Results for structured benchmarks from the DIMACS graph-coloring suite

DIMACS Graph	Nodes	Edges	Has Hamiltonian Cycle?	Total CPU Time [s]		Speedup	Orders of Magnitude Speedup
				Previous	(*ehov*, *f4*, *t4*, *it*)		
myciel7	191	2,360	Yes	>100,000	2.8	>35,714×	4
queen15_15	225	10,360	Yes	23,102	35	660×	2
queen16_16	256	12,640	Yes	>100,000	86	>1,163×	3
miles500	128	1,170	Yes	6,590	2.0	3,295×	3
miles750	128	2,113	Yes	>50,000	1.2	>41,667×	4
miles1000	128	3,216	Yes	>50,000	2.5	>20,000×	4

8 Conclusion

We presented new techniques for relative SAT encoding of permutations with constraints, resulting in improved scalability compared to the previous translation [21] when applied to HCPs. We achieved 3 orders of magnitude average speedup on satisfiable random graphs from the phase transition region, 2 orders of magnitude average speedup on unsatisfiable random graphs, and up to 4 orders of magnitude speedup on satisfiable structured graphs from the DIMACS graph coloring instances. We believe that these techniques can be used to solve other types of permutations with constraints.

References

[1] Ansótegui, C., del Val, A., Dotú, I., Fernández, C., Manyà, F.: Modeling Choices in Quasigroup Completion: SAT vs. CSP. In: 19th National Conference on Artificial Intelligence, July 2004, pp. 137–142 (2004)

[2] Cadoli, M., Schaerf, A.: Compiling Problem Specifications into SAT. Artificial Intelligence 162(1-2), 89–120 (2005)

[3] Cheeseman, P., Kanefsky, B., Taylor, W.M.: Where the Really Hard Problems Are. In: 12th International Joint Conference on AI (IJCAI 1991), pp. 331–337 (1991)

[4] de Kleer, J.: A Comparison of ATMS and CSP Techniques. In: 11th Int'l. Joint Conference on Artificial Intelligence (IJCAI 1989), August 1989, pp. 290–296 (1989)

[5] DIMACS Graph Coloring Instances, `http://mat.gsia.cmu.edu/COLOR/instances.html`

[6] Eén, N., Sörensson, N.: MiniSat—a SAT solver with conflict-clause minimization. In: Bacchus, F., Walsh, T. (eds.) SAT 2005. LNCS, vol. 3569. Springer, Heidelberg (2005)

[7] Frank, J., Martel, C.U.: Phase Transitions in the Properties of Random Graphs. In: Principles and Practice of Constraint Programming, CP 1995 (1995)

[8] Frank, J.: Personal Communication (April 2009)

[9] Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, New York (1979)

[10] Goldberg, E., Novikov, Y.: BerkMin: A Fast and Robust Sat-Solver. In: Design, Automation, and Test in Europe (DATE 2002), March 2002, pp. 142–149 (2002)

[11] Gomes, C.P., Shmoys, D.B.: The Promise of LP to Boost CSP Techniques for Combinatioral Problems. In: International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems (CP-AI-OR 2002), March 2002, pp. 291–305 (2002)
[12] Hnich, B., Walsh, T., Smith, B.M.: Dual Modelling of Permutation and Injection Problems. Journal of Artificial Intelligence Research (JAIR) 21, 357–391 (2004)
[13] Hoos, H.H.: SAT-Encodings, Search Space Structure, and Local Search Performance. In: Int'l. Joint Conference on Artificial Intelligence (IJCAI 1999), August 1999, pp. 296–303 (1999)
[14] Huang, J.: A Case for Simple SAT Solvers. In: Bessière, C. (ed.) CP 2007. LNCS, vol. 4741, pp. 839–846. Springer, Heidelberg (2007)
[15] Iwama, K., Miyazaki, S.: SAT-Varible Complexity of Hard Combinatorial Problems. In: IFIP 13th World Computer Congress, August–September 1994, vol. (1), pp. 253–258 (1994)
[16] Kautz, H.A., Ruan, Y., Achlioptas, D., Gomes, C.P., Selman, B., Stickel, M.E.: Balance and Filtering in Structured Satisfiable Problems. In: 17th Int'l. Joint Conference on Artificial Intelligence (August 2001)
[17] Kautz, H.A., Selman, B.: The State of SAT. Discrete Applied Mathematics 155(12) (June 2007)
[18] Kjaerulff, U.: Triangulation of Graphs—Algorithms Giving Small Total State Space. Technical Report R 90-09, Institute for Electronic Systems, Denmark (March 1990)
[19] Moskewicz, M.W., Madigan, C.F., Zhao, Y., Zhang, L., Malik, S.: Chaff: Engineering an Efficient SAT Solver. In: 38th Design Automation Conference (DAC 2001) (June 2001)
[20] Pipatsrisawat, K., Darwiche, A.: A New Clause Learning Scheme for Efficient Unsatisfiability Proofs. In: Twenty-Third AAAI Conference on Artificial Intelligence (AAAI 2008), July 2008, pp. 1481–1484 (2008)
[21] Prestwich, S.D.: SAT Problems with Chains of Dependent Variables. Discrete Applied Mathematics 130(2), 329–350 (2003)
[22] Rose, D.: Triangulated Graphs and the Elimination Process. Journal of Mathematical Analysis and Applications 32, 597–609 (1970)
[23] Rose, D.: A Graph-Theoretic Study of the Numerical Solution of Sparse Positive Definite Systems of Linear Equations. In: Read, R.C. (ed.) Graph Theory and Computing, pp. 183–217. Academic Press, NY (1973)
[24] Ryan, L.: Siege SAT Solver v.4, `http://www.cs.sfu.ca/~loryan/personal/`
[25] Sato, N., Tinney, W.F.: Techniques for Exploiting the Sparsity or the Network Admittance Matrix. IEEE Transactions on Power Apparatus and Systems 82(69), 944–950 (1963)
[26] Tinney, W.F., Walker, J.W.: Direct Solutions of Sparse Network Equations by Optimally Ordered Triangular Factorization. Proceedings of the IEEE 55(11), 1801–1809 (1967)
[27] Vandegriend, B., Culberson, J.: The $G_{n,m}$ Phase Transition Is Not Hard for the Hamiltonian Cycle Problem. Journal of Artificial Intelligence Research (JAIR) 9, 219–245 (1998)
[28] Velev, M.N., Gao, P.: Efficient SAT-Based Techniques for Design of Experiments by Using Static Variable Ordering. In: 10th International Symposium on Quality Electronic Design (ISQED 2009) (March 2009)
[29] Velev, M.N., Gao, P.: Efficient SAT Techniques for Absolute Encoding of Permutation Problems: Application to Hamiltonian Cycles. In: 8th Symposium on Abstraction, Reformulation and Approximation (2009)
[30] Yanakakis, M.: Computing the Minimum Fill-in Is NP-Complete. SIAM Journal of Algebraic and Discrete Mathematics 2, 77–79 (1981)